LABORATORIO 05

Apache Spark

Cuadro comparativo de RDD, DataFrame y Dataset, con dos demostraciones y una medición de la misma consulta por tres caminos sobre IMDb

Asignatura [asignatura] · Docente [docente] · Integrantes [integrante 1], [integrante 2] · Entrega [fecha]

Objetivo y por qué IMDb

- Comparar RDD, DataFrame y Dataset con evidencia propia, no solo con la documentación
- Construir dos demostraciones ejecutables que impriman el nombre de cada función
- Medir la misma consulta por tres caminos y separar el efecto del motor del efecto del formato
- Documentar los hallazgos en un informe de 19 páginas y exponer en 10 minutos

IMDb ofrece 12,9 millones de títulos y 1,7 millones de calificaciones sin clave ni registro. El filtro de mil votos deja 71.355 títulos de 9 tipos en 16 décadas.

Cuadro comparativo

Criterio	RDD	DataFrame	Dataset
Nivel	Colección de objetos	Tabla con esquema	Tabla tipada
Tipado	Dinámico	En ejecución	Estático en JVM
Optimización	Ninguna	Catalyst y Tungsten	Catalyst y codificadores
Serialización	Java o Kryo	Binaria de Tungsten	Binaria guiada
En Python	Sí	Sí	No
Medición	51.124 ms	2.714 ms / 1.357 ms	No aplica

> Dataset no existe en Python. La API tabular de PySpark es DataFrame, equivalente a un Dataset[Row] sin tipado estático. El error de nombre de columna se descubre en ejecución, no en compilación.

Dataset en Scala

El fragmento se ejecuta en `.\install\run.ps1 shell` y muestra la única diferencia real, que es el momento en que se descubre un error de tipo.

```
case class Title(tconst: String, titleType: String, startYear: Int, averageRating: Double)

val titles = spark.read.parquet("data/curated/imdb/titles").as[Title]

titles.filter(t => t.startYear >= 2000 && t.averageRating >= 8.0)
      .groupBy("titleType")
      .avg("averageRating")
```

Demostración con RDD

Cuatro transformaciones y cuatro acciones que no aparecen en las listas de clase. La demo lee el TSV curado como texto y encadena las ocho sobre pares clave valor.

Transformaciones	Acciones
mapPartitionsWithIndex	takeOrdered
distinct	aggregate
partitionBy	countByValue
join	foreachPartition

map y filter se usan solo para parsear y limpiar, no cuentan como las funciones nuevas.

RDD, salida real

```
[rdd] leidos 71355 titulos y 1727597 notas
[rdd] titles ocupa 8 particiones con [8883, 9014, 8897, 8897, 8884, 8924, 8885, 8971]
[rdd] distinct deja 9 tipos de titulo
[rdd] partitionBy ocupa 8 particiones con [8918, 9025, 9004, 8923, 8906, 8721, 9039, 8819]
[rdd] join empareja 71355 pares en 39.5 s
[rdd] takeOrdered devuelve los 10 titulos con mas votos
[rdd]   The Shawshank Redemption               9.3  3260058
[rdd]   The Dark Knight                        9.1  3245703
[rdd]   Inception                              8.8  2885372
[rdd] aggregate calcula la nota media ponderada 7.152 sobre 1503735096 votos
[rdd] demo terminada en 121.3 s
```

El hallazgo del join

> En PySpark, partitionBy no ahorra el intercambio del join posterior. join se traduce en python_join, que une las dos ramas y las agrupa con groupByKey. Ese groupByKey crea un particionador nuevo y descarta el anterior.

- En Scala, PairRDDFunctions.join compara los particionadores y evita el segundo intercambio
- cogroup también pasa por groupByKey, así que no es una alternativa
- La alternativa práctica es quitar el partitionBy y dejar que el join haga el suyo

La comprobación está en pyspark/join.py, función _do_python_join, no en la intuición.

Demostración con DataFrame

Cinco funciones de clase y cuatro investigadas. Cada bloque imprime el nombre y su resultado. El plan físico se muestra solo en agg.

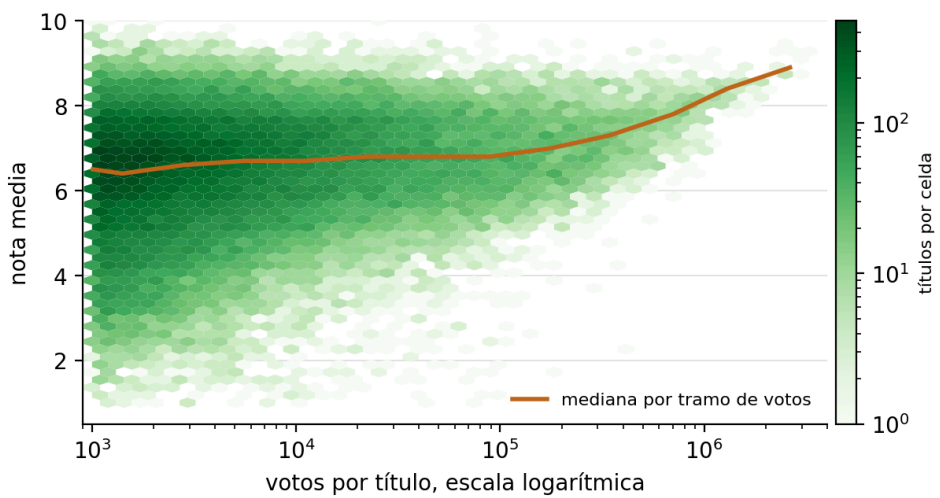
Clase	Investigadas
select	Window con row_number
filter	pivot
groupBy	explode con split
agg	regexp_replace
orderBy	

corr y percentile_approx se usan como refuerzo, sin reemplazar a las cuatro investigadas.

DataFrame, salida real

```
[dataframe] leidos 71355 titulos con nota
[dataframe] filter deja 10342 peliculas desde 1990 con diez mil votos o mas
[dataframe] groupBy produce 346 combinaciones de genero y decada
[dataframe] agg calcula nota media, votos y titulos por genero
[dataframe] orderBy desempata la nota por cantidad de votos
[dataframe] Window con row_number deja tres titulos por genero
[dataframe] pivot cruza la decada con tres tipos de titulo
[dataframe] regexp_replace encuentra 4984 titulos repetidos
[dataframe] corr sin filtrar nulos da -0.129 porque Spark los lee como cero
[dataframe] corr entre duracion y nota da -0.056 sobre 68047 titulos
[dataframe] demo terminada en 15.9 s
```

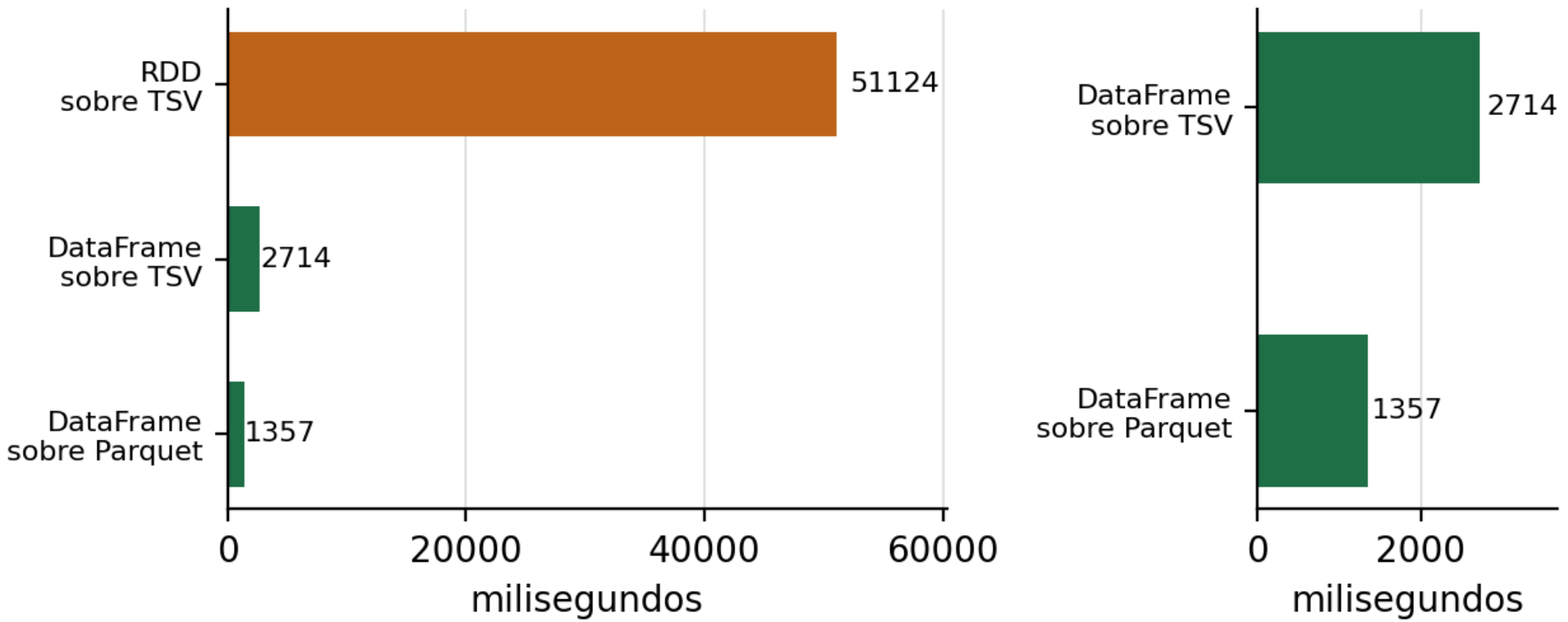
Nota y popularidad



- Con pocos votos la nota se dispersa entre 1 y 10
- Basta un puñado de calificaciones altas para llegar al 9,9
- El podio por nota se llena de títulos oscuros
- La mediana sube en los títulos con muchos votos

> Ordenar por nota exige un mínimo de votos. numVotes se conserva en el esquema para poder hacer esa distinción.

Medición de las tres rutas



RDD sobre TSV 51.124 ms · DataFrame sobre TSV 2.714 ms · DataFrame sobre Parquet 1.357 ms

Interpretación

- **Interfaz.** RDD transporta cada línea como cadena de Python y el motor no puede inspeccionarla. DataFrame lee con un analizador en la JVM y mueve datos binarios. Diferencia de casi diecinueve veces.
- **Formato.** Parquet guarda tipos, comprime por columnas y permite leer solo las columnas necesarias. La consulta toca tres de diez, y el ahorro es directo.
- **Independencia.** Las dos mejoras se acumulan. Pasar de RDD sobre texto a DataFrame sobre Parquet va de 51 segundos a poco más de uno.

Dificultades

- **Canal del trabajador.** first() sobre particiones grandes corta la conexión en Windows. Se descarta la cabecera en el analizador.
- **Memoria.** Doce núcleos con un giga de heap reparten tan poco que el trabajo aborta. Se baja a local[2] con ocho particiones.
- **Módulos.** src no llega a los trabajadores sin addPyFile.
- **Nulos.** Correlation lee los nulos como cero y devuelve un número equivocado sin avisar.

Conclusiones

- La interfaz pesa más que el formato
- El formato se suma a la interfaz
- El RDD sigue teniendo sentido sin esquema o con particionador propio, pero en Python no sobrevive entre partitionBy y join
- El DataFrame no elimina la necesidad de pensar, ordenar por nota llena el podio de títulos oscuros
- Las dificultades aparecen solo sobre datos reales y se resuelven con cambios pequeños
- La reproducibilidad no es un adorno

Cierre

El cuadro comparativo, las dos demostraciones y la medición cubren los cinco puntos del enunciado.

La medición añade una fila que no estaba pedida y que resulta ser la más útil, porque obliga a separar dos efectos que suelen presentarse juntos.

Repositorio, código y evidencia en https://github.com/nemryz/spark-lab05